AGRISURGE

AI-Powered Agricultural Risk & Dynamic Insurance Pricing Platform

Subject Mapping Report

Python Programming for Data Science

Database Management

October 2026

1. Introduction

AgriSurge is an enterprise-grade underwriting and actuarial risk platform built for agricultural micro-insurance. It combines satellite earth observation (Sentinel NDVI), IoT ground sensor telemetry (soil moisture, ambient temperature, relative humidity), 56 years of historical district agricultural records (ICRISAT 1966–2013), modern Government Agricultural Production & Yield data (APY 2014–2023 across 35 Maharashtra districts), and daily 0.25° gridded rainfall datasets (IMD 2015–2025). The system predicts crop-failure risk and harvest yields, computes explainable risk factors, and executes actuarially sound dynamic insurance premium pricing.

This report maps the project to two subjects: **Python Programming for Data Science** and **Database Management**. For each subject it identifies the concepts applied, the module in which each is implemented, and the empirical results obtained from the evaluation harness. The report is based on the final project source code, Drizzle ORM PostgreSQL schema, machine learning training pipelines, and evaluation benchmarks.

1.1 System overview

Multimodal data is collected from IMD gridded NetCDF archives, Government APY HTML/XLS exports, ICRISAT district time-series, and real-time IoT hardware (ESP32 microcontrollers). The data is parsed, harmonized into uniform metric units (Kg/ha), and stored in a cloud PostgreSQL database hosted on Neon Serverless. A high-performance FastAPI microservice serves trained Scikit-Learn pipelines (Ridge Regression for yield prediction, ExtraTrees for 22-class crop suitability) and pure NumPy decision trees. Actuarial pricing is calculated through a decoupled mathematical engine. An interactive Next.js 16 dashboard provides GIS polygon boundary mapping (Mapbox GL), underwriting audit workflows, and automated insurance policy certificate generation.

2. Subject-wise Summary

Subject	Where it is used in AgriSurge	Key files
Database Management (DBMS)	PostgreSQL via Neon Serverless and Drizzle ORM with 7 normalized relational tables (3NF), primary/foreign key integrity, candidate business keys with UNIQUE constraints, domain-specific PostgreSQL Enums, JSONB semi-structured storage for GeoJSON boundaries and audit trails, high-precision numeric currency/coordinates, parameterized type-safe SQL queries, 5-table relational joins, and dual-layer fallback persistence.	`db/schema.ts` `db/index.ts` `lib/underwriting/underwritingStore.ts` `app/api/iot/telemetry/route.ts`
Python Programming for Data Science (PDS)	Multi-source heterogeneous ingestion (NetCDF daily weather, APY XLS, ICRISAT CSV, 22-crop NPK dataset, IoT streams); spatial raster grid aggregation over Maharashtra coordinates; temporal crop-marketing to calendar year alignment; feature engineering (lag yields, 5-yr rolling mean, 14 IMD weather features); ML modeling (Ridge, RandomForest, ExtraTrees, GradientBoosting, Pure NumPy DecisionTree); model evaluation (MAE, RMSE, R^2=0.9253, 99.39% accuracy); explainable AI permutation importance; joblib/pickle artifact serialization; and asynchronous FastAPI microservice with Pydantic v2 schemas.	`ml/train.py, ml/model.py` `ml/data_analysis.py` `ml/evaluate.py` `ml-service/app/main.py` `ml-service/app/predictor.py` `ml-service/app/schemas.py`

3. Database Management (DBMS)

Database: Cloud PostgreSQL hosted on Neon Serverless, managed declaratively with Drizzle ORM. The relational schema contains **7 core entities** structured in 3rd Normal Form (3NF). It supports high-precision decimal storage, domain constraints, JSONB spatial geometry, and comprehensive audit trails.

3.1 Database schema

Table	Purpose	Key / constraint	Attributes & Types
farmers	Master entity storing personal and contact details of registered agricultural owners	PK: id (serial)	`name (varchar 120, not null), phone (varchar 20), region (varchar 80), created_at (timestamp)`
farms	Core farm parcel storing crop species, soil classification, area, and geospatial boundaries	PK: id (serial); UNIQUE(farm_code); FK: farmer_id -> farmers.id	`farm_code (varchar 20), area_acres (numeric 8,2), boundary_geojson (jsonb), latitude/longitude (numeric 9,6)`
weather_data	Time-series environmental records and real-time IoT hardware telemetry for a parcel	PK: id (serial); FK: farm_id -> farms.id	`temperature_c (numeric 5,2), rainfall_mm (numeric 6,2), humidity_pct, wind_speed_kph, status (enum)`
risk_predictions	Machine learning inference scores, crop-failure probability, and explainable AI factors	PK: id (serial); FK: farm_id -> farms.id	`risk_score (numeric 4,3), risk_level (enum: low/mod/high), factors (jsonb), model_name, is_mock (bool)`

Table	Purpose	Key / constraint	Attributes & Types
premium_predictions	Actuarial dynamic quotes derived mathematically from ML risk assessment	PK: id (serial); FK: risk_prediction_id -> risk_predictions.id (1:1)	`base_premium (numeric 10,2), multiplier (numeric 4,2), recommended_premium (numeric 10,2), created_at`
policies	Underwriting insurance contracts, review status, and historical immutable snapshot	PK: id (serial); UNIQUE(policy_code); FK: farm_id; FK: premium_prediction_id	`coverage_amount (numeric 12,2), status (varchar 40), assessment_snapshot (jsonb), audit_trail (jsonb), certificate_pdf`
notifications	Automated agronomic alerts, weather warnings, and underwriting flags	PK: id (serial); FK: farm_id -> farms.id (nullable)	`severity (enum: high/medium/info), message (text), read (boolean), created_at (timestamp)`

3.2 DBMS concepts applied

DBMS concept	How it appears in the project	File
Schema design / DDL	Strongly-typed declarative DDL definitions via Drizzle ORM (pgTable, serial, varchar, numeric, timestamp, jsonb, pgEnum) guaranteeing zero schema drift.	`db/schema.ts`
Primary & Candidate keys	Surrogate auto-incrementing primary keys (id: serial()) alongside business candidate keys with UNIQUE constraints (farmCode: unique(), policyCode: unique()).	`db/schema.ts`
Referential integrity & Foreign keys	Relational foreign key constraints enforce relational parentage: 1:M (farms.farmerId -> farmers.id), 1:M (weather_data.farmId -> farms.id), and 1:1 (premium_predictions.riskPredictionId -> risk_predictions.id).	`db/schema.ts`
Database Normalization (3NF)	Strict 3rd Normal Form architecture: farmers, farm plots, time-series observations, ML inference scores, and policy contracts reside in distinct entities, preventing update, deletion, and insertion anomalies.	`db/schema.ts`
Domain integrity & Enumerations	PostgreSQL ENUM types restrict column values to legal domain sets: risk_level ('low', 'moderate', 'high'), policy_status ('active', 'pending', 'expired', 'under_review'), and alert_severity.	`db/schema.ts`
Numeric precision & Scale	Exact-precision decimal types eliminate floating-point rounding errors: numeric(12,2) for policy sum insured (up to ■999 Crore), numeric(4,2) for risk multipliers, and numeric(9,6) for sub-meter GPS coordinates.	`db/schema.ts`
Semi-structured data (JSONB)	Hybrid relational/document storage: boundaryGeoJson stores complex polygon coordinates; factors stores dynamic explainable AI feature contribution lists; auditTrail stores chronological event arrays.	`db/schema.ts`
Parameterized queries & SQL injection safety	All queries use Drizzle ORM query builder compiling to parameterized SQL placeholders ($1, $2), completely preventing SQL injection vulnerabilities.	`db/index.ts, underwritingStore.ts`
Relational joins & Query composition	Multi-table relational queries: leftJoin across 5 tables simultaneously (policies ■ farms ■ farmers ■ premium_predictions ■ risk_predictions) using primary/foreign key joins.	`lib/underwriting/ underwritingStore.ts`
Audit logging & Historical immutability	assessmentSnapshot freezes farm environmental data, ML risk score, and premium recommendation at the exact time of approval, ensuring legal policy records remain immutable over time.	`lib/underwriting/ underwritingStore.ts`
Connection management & Serverless pooling	Neon HTTP driver (@neondatabase/serverless) establishes stateless SQL execution over HTTPS, avoiding traditional TCP connection exhaustion during high-concurrency serverless execution.	`db/index.ts`
Data seeding & Fault-tolerant fallback	Dual-layer persistence design: seamlessly reads/writes from Neon PostgreSQL when available, with automatic fallback to local JSON persistence (underwriting_records.json) during offline development.	`lib/underwriting/ underwritingStore.ts`

Summary. The database tier utilizes PostgreSQL via Neon Serverless and Drizzle ORM across 7 normalized tables. Every write is strongly validated by Drizzle schema constraints, primary/candidate keys guarantee unique identification, relational joins aggregate multi-table underwriting applications into a single round-trip, and JSONB fields enable semi-structured storage for GIS parcel polygons and immutable underwriter audit snapshots.

4. Python Programming for Data Science (PDS)

Python serves as the complete data science backbone of AgriSurge. It orchestrates multi-source agricultural ingestion, multidimensional gridded NetCDF climate processing, statistical data wrangling, lag/rolling feature engineering, supervised machine learning model training (Ridge Regression, ExtraTrees, RandomForest, GradientBoosting), and serves real-time inference via an asynchronous FastAPI microservice with Pydantic v2 schemas.

4.1 Data science concepts applied

PDS concept	How it appears in the project	File
Multi-source data ingestion	Parsing heterogeneous scientific data: NetCDF-4 daily rainfall grids (RF25_ind{2015..2025}_rfp25.nc), multi-year HTML/XLS tables (Government APY), historical district CSVs (ICRISAT), and 2,200 soil-crop samples.	`ml/train.py`, `ml/data_analysis.py`
Data wrangling & cleaning	Pandas DataFrame manipulation: un-pivoting 147 metric columns across 35 Maharashtra districts, harmonizing district reorganizations (Palghar carved from Thane, Hingoli from Parbhani), and dropping corrupt records.	`ml/data_analysis.py`, `ml/preprocessing.py`
Unit harmonization	Standardizing historical yield units: converting Indian cotton bales (170 Kg lint per bale) and agricultural tonnes into uniform kilograms per hectare (yield_kg_ha). No synthetic binary targets were fabricated.	`ml/train.py`, `ML_V2_TRAINING_REPORT.md`
Temporal agricultural alignment	Deterministic temporal mapping between Indian agricultural marketing crop years (YYYY - YYYY+1) and IMD weather calendar years: Kharif (monsoon) and Rabi (winter) aligned to weather starting year YYYY.	`ml/train.py`, `ML_V2_TRAINING_REPORT.md`
Spatial grid raster extraction	Subsetting daily 0.25° gridded rainfall NetCDF over Maharashtra bounding box (15.5°–22.1°N, 72.5°–80.5°E), aggregating 891 grid cells daily across 11 calendar years (2015–2025) into regional rainfall distributions.	`ml/train.py`
Lag & rolling feature engineering	Constructing agronomic predictors: 1-year yield lag (prev_yield_kg_ha), 2-year yield lag (prev2_yield_kg_ha), 1-year area lag (prev_area_1000ha), and 5-year rolling mean yield (hist5y_mean_yield).	`ml/train.py`, `ML_V2_TRAINING_REPORT.md`
Meteorological feature engineering	Extracting 14 weather features from daily IMD rainfall: total annual, Kharif/Rabi sums, rainy days (≥2.5mm), dry days, peak single-day rain, and critical pre-harvest rainfall windows (7d, 30d, and 90d cumulative rainfall before cutoff).	`ml/train.py`
Exploratory data analysis & Statistics	Automated statistical summary generation via df.describe().T, missingness tracking, Pearson correlation matrices with target (corr()['crop_failure']), and class distribution checks across 22 crops.	`ml/data_analysis.py`, `ml/dataset_report.txt`
Data models & Validation	Strict data validation using Pydantic v2 schemas: RiskPredictionInput, RiskPredictionOutput, and RiskFactor enforcing field types, bounding constraints, and alias conversions.	`ml-service/app/schemas.py`

PDS concept	How it appears in the project	File
Machine learning pipelines	Scikit-Learn Pipeline encapsulating StandardScaler, SimpleImputer, OrdinalEncoder, and estimators (Ridge, RandomForest, GradientBoosting, ExtraTreesClassifier).	`ml/train.py`, `ml-service/app/predictor.py`
Chronological vs Stratified split	Strict chronological splitting for yield time series (Train: 1967–2014 [12,358], Val: 2015–2019 [1,663], Test: 2020–2022 [1,024]) avoiding look-ahead leakage; Stratified 70/15/15 split for 22-class crop recommendation.	`ml/train.py`, `ml/evaluate.py`
Multi-algorithm comparison	Comparing 5 classification architectures (ExtraTrees, RandomForest, HistGradientBoosting, GradientBoosting, KNN) and 4 regression models (Ridge, RandomForest, GradientBoosting, HistGradientBoosting).	`ml/train.py`, `ML_V2_TRAINING_REPORT.md`
Explainable AI & Feature importance	Permutation importance (ΔR^2) measuring real feature contribution on held-out test sets; Decision tree variance reduction gain; dynamically mapping top contributing factors (NDVI stress, moisture deficit, heat index) into inference JSON.	`ml-service/app/predictor.py`, `ml/model.py`
Model serialization & Artifacts	Persisting trained estimators and pipelines via joblib (.joblib) and pickle (.pkl) with associated metadata JSON files documenting feature names, training dates, and hyperparameters.	`ml/models/*`, `ml-service/model/*`
Asynchronous web service API	FastAPI service with /predict-risk endpoint, CORS middleware, lifespan management, and Uvicorn ASGI server. Connects Next.js frontend to Python inference engine with microsecond latency.	`ml-service/app/main.py`, `ml-service/app/predictor.py`
Real-time IoT stream ingestion	RESTful ingestion of ESP32 sensor telemetry (soil moisture, temperature, humidity), sensor raw-to-percent conversion, and out-of-bounds telemetry validation (NORMAL, CAUTION, LOW flags).	`app/api/iot/telemetry/route.ts`, `lib/iot/iotStore.ts`

4.2 Machine learning architectures & Mathematical formulation

AgriSurge employs three distinct machine learning model implementations tailored for agricultural underwriting:
Model A: Crop Failure Risk & Yield Model (V2). Trained on 15,045 historical records (1966–2022) across 37 Maharashtra districts and 12 target crops (Cotton, Rice, Wheat, Sugarcane, Sorghum, Soybean, Maize, etc.). Combines agricultural lag features and 14 IMD weather metrics.
Model B: 22-Class Crop Recommendation Engine. Multi-class classifier consuming 7 soil-climatic features (N, P, K, Temp, Humidity, pH, Rainfall) across 2,200 samples. Produces calibrated softmax class probability vectors for Top-3 crop recommendations.
Model C: Pure NumPy Decision Tree. Algorithmic reference implementation built from scratch using pure NumPy matrix operations (depth=6, variance reduction splits).

Mathematical & Actuarial Formulation

1. Agronomic Yield Deviation & ML Crop-Failure Risk:

```
Yield_Deviation (%) = ((Expected_Yield - Predicted_Yield) / Expected_Yield) * 100
```

```
Risk_Score = min(0.97, max(0.05, 0.35 * Rainfall_Deficit + 0.25 * Heat_Stress + 0.20 * Moisture_Deficit + 0.20 * NDVI_Stress))
```

2. Actuarial Dynamic Premium Calculation (Decoupled Pricing Engine):

```
Base_Exposure = Area_Acres * Crop_Base_Rate (INR/Acre)
```

```
Risk_Multiplier = 1.0 + (Risk_Score * 0.75) + Climate_Surcharge
```

```
Recommended_Premium = Base_Exposure * Risk_Multiplier
```

Summary. Python is used to ingest multi-source scientific data, align marketing years with meteorological calendars, engineer agricultural lag and climate stress features, train supervised regression and classification models, and expose real-time inference through an asynchronous FastAPI microservice. Model predictions are strictly decoupled from the actuarial pricing engine.

4.3 Quantitative evaluation results & benchmarks

All models in AgriSurge are rigorously evaluated on strictly held-out test partitions. The tables below document the empirical performance benchmarks across the three machine learning systems.

Agricultural Yield Prediction Benchmark (Chronological Test Set: 2020–2022, 1,024 Rows)

Experiment / Model Architecture	Test MAE (Kg/ha)	Test RMSE (Kg/ha)	Test R^2 Score
Baseline DummyRegressor (Mean)	6,333.36	21,817.37	-0.0660
RandomForest Regressor (Agri + Weather)	4,242.33	15,343.90	0.4727
GradientBoosting Regressor (Agri + Weather)	3,208.94	12,105.15	0.6718
HistGradientBoosting (Agri + Weather)	5,033.24	18,163.76	0.2611
Ridge Regression Pipeline [Selected Winner]	**1,534.91**	**5,773.46**	**0.9253**

22-Class Crop Recommendation Model Comparison (Held-out Test Set: 330 Samples)

Model Algorithm	Val Accuracy	Val Macro F1	Test Accuracy	Test Macro F1
ExtraTreesClassifier [Winner]	**0.9939**	**0.9939**	**0.9939 (99.39%)**	**0.9939 (99.39%)**
RandomForestClassifier	0.9909	0.9909	0.9909	0.9909
HistGradientBoosting	0.9879	0.9878	0.9879	0.9878
GradientBoostingClassifier	0.9848	0.9848	0.9848	0.9848
KNeighborsClassifier (k=5)	0.9848	0.9848	0.9848	0.9848

Pure NumPy Decision Tree & Explainable AI Benchmarks

Model / Evaluation Dimension	Formula / Methodology	Quantitative Result
Pure NumPy Tree — Accuracy	(TP + TN) / Total (Test n = 44)	**0.9318 (93.18%)**
Pure NumPy Tree — Precision & Recall	Precision = TP / (TP + FP); Recall = TP / (TP + FN)	Precision: **0.8889** | Recall: **0.9412**
Pure NumPy Tree — F1 Score	2 * (Precision * Recall) / (Precision + Recall)	**0.9143 (91.43%)**
Permutation Importance: prev_yield_kg_ha	ΔR^2 drop on held-out test when shuffled	**0.6092** (Std: 0.0236) — Top predictor
Permutation Importance: hist5y_mean_yield	ΔR^2 drop on held-out test when shuffled	**0.3960** (Std: 0.0183) — Baseline trend
Tree Gini Gain: ndvi	Variance reduction gain across tree splits	**0.6788 (67.88%)** — Vegetative health
Tree Gini Gain: temperature_c & Moisture	Variance reduction gain across tree splits	Temp: **0.1262** | Soil Moisture: **0.1240**

5. Data Flow and Subject Integration

The following diagram illustrates how multimodal agricultural and climate data flows through AgriSurge, highlighting the seamless interface between **Python Programming for Data Science** and **Database Management**.

```
IMD Gridded NetCDF / Govt APY (XLS) / ICRISAT (CSV) / Crop NPK / IoT Sensors
 --(PDS: xarray NetCDF spatial raster extraction, 891 grid cells)-->
 --(PDS: Pandas unpivoting 147 cols, district normalization, imputation)-->
 --(PDS: lag feature engineering: 1y/2y yield, 5y rolling mean)-->
 |
 v
Neon PostgreSQL (7 tables in 3NF via Drizzle ORM)
 --(DBMS: type-safe DDL, primary/candidate keys, domain enums, numeric 9,6)-->
 --(DBMS: jsonb GeoJSON parcel boundaries, IoT weather observations)-->
 |
 v
Machine Learning Models: Ridge Regression (V2) | ExtraTrees (22 Crops)
 --(PDS: chronological train/val/test splits, StandardScaler, SimpleImputer)-->
 --(PDS: yield prediction R2 = 0.9253, crop recommendation F1 = 99.39%)-->
 |
 v
FastAPI Microservice (Uvicorn ASGI + Pydantic v2 schemas)
 --(PDS: /predict-risk async endpoint, explainable AI risk factors, XAI)-->
 |
 v
Actuarial Pricing Engine & Underwriting Contracts
 --(DBMS: relational 5-table JOIN across policies, farms, farmers, quotes)-->
 --(DBMS: dynamic risk multiplier quotes, immutable assessmentSnapshot)-->
 |
 v
Underwriting Dashboard (Next.js 16 + Mapbox GL) & Policy Certificates
 --(Full-Stack: interactive parcel polygons, auditTrail logs, certificate PDF)-->
```

6. Conclusion

AgriSurge establishes a cohesive, robust, and mathematically grounded synergy between **Database Management** and **Python Programming for Data Science**:

• **Database Management Systems (DBMS)** principles provide an ACID-compliant, highly normalized 3NF relational architecture hosted on Neon Serverless PostgreSQL and enforced through Drizzle ORM. It guarantees referential integrity across farmers, agricultural parcels, IoT weather streams, and underwriting contracts. The platform leverages candidate business keys, database-level domain enumerations, arbitrary-precision currency and geospatial decimals (numeric 12,2 / 9,6), hybrid JSONB storage for GIS parcel polygons, parameterized query execution for SQL injection protection, and immutable underwriting snapshots that ensure lifelong policy auditability.

• **Python Programming for Data Science (PDS)** delivers the end-to-end analytical and machine learning intelligence: ingesting multidimensional NetCDF raster grids and long-term agricultural archives; harmonizing metric crop units; aligning Indian crop marketing seasons with meteorological calendars; engineering historical lags and 14 meteorological stress indicators; training supervised regression and multi-class classification pipelines with strict chronological validation (R2 = 0.9253, Crop Suitability F1 = 99.39%); and serving low-latency explainable predictions via an asynchronous FastAPI microservice with Pydantic v2 schemas.

Together, the integration of these two core computer science disciplines enables AgriSurge to bridge the gap between raw environmental observations and transparent, data-driven agricultural insurance underwriting.